

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Disciplina: Testes de Software

Trabalho: Detecção de Bad Smells e Refatoração Segura

Aluno: João Pedro Aguiar do Prado

Matrícula: 756491

Data: 10/05/2026



1. Análise de Smells

A análise manual do código de produção original ([src/ReportGenerator.js](#)) revelou graves deficiências estruturais que comprometem a manutenibilidade e a testabilidade do sistema. Foram identificados três *Bad Smells* principais:

1. Método Longo (Long Method): O método `generateReport` concentra múltiplas responsabilidades. Ele gerencia a instanciação de variáveis, a lógica de filtragem de permissões baseada no cargo do usuário, o cálculo de totais e a construção de strings para diferentes formatos de saída (CSV e HTML).

- **Por que é problemático:** Fere diretamente o Princípio da Responsabilidade Única (SRP). Um método extenso aumenta drasticamente a carga cognitiva de quem o lê. Em um cenário real de engenharia, métodos com essa característica inflam a latência nas revisões de código (*Pull Requests*), pois o revisor precisa mapear mentalmente diversas regras de negócios simultâneas. Além disso, testar uma regra de formatação HTML exige a configuração de todo o escopo de permissões, dificultando testes unitários isolados.

2. Complexidade Condicional Aninhada: A iteração principal sobre os itens contém múltiplos níveis de `if/else if`, validando sequencialmente regras de negócio e tipos de relatório de forma acoplada.

- **Por que é problemático:** Cria um emaranhado lógico de difícil navegação. A ramificação excessiva obscurece o fluxo principal da aplicação e aumenta a chance de introdução de *bugs* silenciosos ao se adicionar uma nova condicional, além de exigir um número exponencial de casos de teste para cobrir todos os caminhos possíveis.

3. Código Duplicado (**Duplicated Code**): A lógica de acréscimo ao total da operação (`total += item.value`) foi repetida em três ramificações condicionais distintas.

- **Por que é problemático:** A duplicação espalha uma única regra de negócio por vários pontos do arquivo. Se a forma de calcular o total for alterada futuramente (como a adição de uma taxa ou imposto), o desenvolvedor precisará replicar a

mudança em todos esses pontos, aumentando a fragilidade do sistema para manutenções futuras.

2. Relatório da Ferramenta de Análise Estática

Para validar e quantificar as anomalias detectadas manualmente, a ferramenta ESLint foi configurada em conjunto com o [eslint-plugin-sonarjs](#).

(Insira aqui o Print do terminal mostrando os erros originais - ex: File 2.png contendo "Cognitive Complexity from 27 to the 5 allowed")

A utilização do SonarJS foi fundamental para expor gargalos que a análise manual não consegue medir com precisão. Enquanto a leitura humana identifica que um método é "difícil de ler", a ferramenta quantificou o problema, apontando uma **Complexidade Cognitiva de 27**, muito acima do limite aceitável de 5. A análise estática provou ser indispensável para fornecer métricas claras e imparciais sobre a saúde do código, alertando sobre condicionais que poderiam ser aglutinadas e lógicas sobrepostas que impactam diretamente a eficiência da sustentação do software.

3. Processo de Refatoração

O *smell* mais crítico abordado foi o **Método Longo** do [generateReport](#), associado à sua altíssima complexidade condicional.

Antes da Refatoração (Trecho do código acoplado):

```
// O método original possuía o for-loop com regras de negócio e formatação misturadas
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) { item.priority = true; }
    if (reportType === 'CSV') { /* monta string CSV e soma total */ }
    else if (reportType === 'HTML') { /* monta string HTML e soma total */ }
  } else if (user.role === 'USER') { /* repete lógicas de limite, soma e montagem */ }
}
```

Técnica Aplicada: Foi utilizada a técnica **Extract Method**. O objetivo foi fatiar o método gigante em três responsabilidades isoladas: filtragem, cálculo e apresentação.

Depois da Refatoração (Método principal limpo):

```
generateReport(reportType, user, items) {  
  // 1. Filtragem isolada (Regras de negócio e permissões)  
  const filteredItems = this.filterItems(user, items);  
  
  // 2. Cálculo isolado (Eliminação da duplicação)  
  const total = filteredItems.reduce((sum, item) => sum + item.value, 0);  
  
  // 3. Formatação delegada  
  if (reportType === 'CSV') return this.generateCsv(user, filteredItems, total);  
  if (reportType === 'HTML') return this.generateHtml(user, filteredItems, total);  
  
  return '';  
}
```

Com essa abordagem, o loop condicional complexo foi extraído para o método privado `filterItems`, enquanto a formatação das strings foi movida para `generateCsv` e `generateHtml`. Isso resultou na eliminação completa da complexidade reportada pelo SonarJS e no fim da duplicação da variável de `total`.

4. Conclusão

A execução desta refatoração reforçou a premissa de que alterar código legado sem validação contínua é um risco inaceitável. A utilização do framework Jest como uma rede de segurança garantiu que as abstrações arquiteturais aplicadas não alterassem o comportamento externo da aplicação. Sempre que um novo método era extraído, a suíte de testes validava a integridade do sistema em tempo real.

A eliminação dos *Bad Smells* reduziu drasticamente a complexidade cognitiva do artefato de software. Em um ambiente de desenvolvimento colaborativo, um design limpo e modularizado é essencial não apenas para evitar *bugs*, mas para otimizar a eficiência e a distribuição da contribuição efetiva entre a equipe técnica, reduzindo gargalos de interpretação e acelerando a evolução do produto.